압박 면접 질문 및 정리

> 목적: 공격적으로 들어오는 질문에 대해, 방어가 아니라 `인정 + 이유 + 기준 + 다음 개선` 구조로 답하기 위한 문서;

답변 기본 원칙

모든 압박 질문은 아래 순서로 답한다.;

1. 먼저 약점을 인정한다.;
2. 왜 그런 선택이 나왔는지 프로젝트 맥락을 설명한다.;
3. 그 선택으로 얻은 것과 잃은 것을 같이 말한다.;
4. 다시 한다면 무엇을 먼저 바꿀지 말한다.;

---

1. 접근성/시맨틱이 중요한데 왜 프로젝트는 약한가요?

좋은 답변

맞는 지적입니다. 이 프로젝트는 공개 콘텐츠형 웹사이트보다 로그인 이후 사용하는 앱형 서비스에 가까웠고, 초기에는 인증, 거래 처리, AI 파싱, 데이터 정합성, 모바일 UX 검증에 우선순위를 뒀습니다. 그래서 접근성과 시맨틱이 뒤로 밀린 건 사실입니다. 다만 몰라서 못 한 게 아니라 우선순위의 결과였고, 다시 한다면 시맨틱 구조, 포커스 관리, 확대 허용부터 더 초기에 설계에 넣겠습니다.;

피해야 할 답변

- 앱이라 접근성은 덜 중요했다.;
- 시간 없어서 못 했다.;

---

2. `userScalable: false` 는 접근성을 해치는데 왜 넣었나요?

좋은 답변

앱형 화면 밀도와 폼 입력 안정성을 우선한 선택이었습니다. 다만 접근성 기준에서는 분명 부적절한 결정이라고 생각합니다. 지금이라면 확대 허용을 기본으로 두고, 깨지는 화면은 레이아웃으로 해결하는 방향을 택하겠습니다.;

피해야 할 답변

- 요즘 다 확대 막는다.;
- 디자인이 깨져서 어쩔 수 없었다.;

---

3. 로그인 서비스라 SEO가 덜 중요하다는 건 너무 쉬운 변명 아닌가요?

좋은 답변

로그인 기반 서비스라 SEO 효용이 낮았던 건 맞지만, 그렇다고 0이라고 보진 않습니다. 공개 페이지와 메타데이터 구조는 더 보강했어야 했습니다. 당시에는 투자 대비 우선순위가 낮았지만, 공개 유입이 필요한 단계가 되면 기술 SEO와 정보 구조를 바로 보강하는 것이 맞다고 봅니다.;

피해야 할 답변

- 로그인 서비스는 SEO 필요 없다.;
- SEO는 마케터 영역이다.;

---

4. 시맨틱과 접근성은 왜 처음부터 반영하지 않았나요?

좋은 답변

그 지적은 맞습니다. 뒤에 붙일수록 비용이 커집니다. 다만 당시에는 새 스택 학습과 핵심 기능 검증을 동시에 하고 있었기 때문에, 최소 구조만 먼저 깔고 제품 핵심 흐름을 검증한 뒤 접근성 패스를 돌리는 전략을 택했습니다. 지금 다시 한다면 최소 접근성 기준은 훨씬 초기에 고정하겠습니다.;

피해야 할 답변

- 나중에 붙여도 별 문제 없다고 생각했다.;

---

5. React Query를 안 쓴 건 익숙하지 않아서 피한 것 아닌가요?

좋은 답변

러닝커브를 고려한 것도 사실이지만, 더 큰 이유는 구조였습니다. 이 프로젝트는 조회는 서버 컴포넌트, 수정은 `Server Actions`, 갱신은 `revalidatePath`와 `router.refresh`로 이미 정리돼 있었습니다. 여기서 `React Query`를 넣으면 현재 규모에서는 서버 캐시와 클라이언트 캐시가 이중화될 가능성이 컸습니다. 규모가 커지고 optimistic update나 background refetch가 중요해지면 그때는 부분 도입이 맞다고 생각합니다.;

피해야 할 답변

- Next에서는 React Query가 필요 없다.;
- 귀찮아서 안 넣었다.;

---

6. React Query가 없으면 UX가 약해지는데 그걸 감수한 이유는요?

좋은 답변

맞습니다. 체감 속도와 background refetch 측면에서는 손해가 있습니다. 대신 당시에는 데이터 정합성과 서버 중심 흐름 이해가 더 중요했습니다. 즉각적인 UX 일부를 포기하는 대신 구조 단순성과 캐시 정책 관리 비용 절감을 택한 것입니다.;

---

7. Zustand를 안 쓴 것도 결국 전역 상태 설계를 못 해서 그런 것 아닌가요?

좋은 답변

이 프로젝트는 전역 상태가 거의 없었고, 대부분이 폼 입력값, 시트 열림 여부, 필터, 선택 상태 같은 로컬 UI 상태였습니다. 그래서 작은 `Context` 와 로컬 상태로 충분하다고 판단했습니다. 다만 크로스페이지 draft, 복수 패널 동기화 같은 요구가 생기면 그때는 `Zustand` 를 검토하는 게 맞습니다.;

피해야 할 답변

- 작은 프로젝트에선 상태관리가 필요 없다.;

---

8. 왜 굳이 App Router와 Server Actions를 택했나요?

좋은 답변

로그인 기반 서비스였고, 거래와 통계, 예산, 자산처럼 서로 연결된 데이터가 많아서 서버 중심 흐름이 잘 맞았습니다. `Server Actions` 를 쓰면 인증, 권한 검증, DB 접근을 서버 경계 안에 두기 쉽고, mutation 이후 재검증 흐름도 단순했습니다. 개발 속도와 데이터 은닉, 정합성을 함께 고려한 선택이었습니다.;

---

9. Server Actions가 곧 보안은 아닌데, 보안적으로 정확히 무슨 이점이 있었나요?

좋은 답변

맞습니다. 숨긴다고 자동으로 안전해지는 건 아닙니다. 다만 인증 체크, 권한 검증, DB 접근, 민감 로직을 클라이언트 번들에 노출하지 않고 서버 경계에서 일관되게 검증할 수 있다는 점이 구조적 이점이었습니다. 보안은 결국 세션 처리, 입력 검증, user scope 확인이 함께 가야 한다고 생각합니다.;

---

10. 백엔드와 DB가 처음인데 왜 PostgreSQL과 Supabase를 골랐나요?

좋은 답변

거래, 카테고리, 자산, 예산, 통계처럼 관계와 집계가 중요한 도메인이기 때문에 관계형 DB가 자연스러웠습니다. 그리고 입문자일수록 불확실성이 큰 선택보다 자료가 많고 검증된 조합이 필요하다고 봤습니다. `Supabase`는 운영 부담을 줄이면서 `PostgreSQL` 기반 구조를 익히기에 현실적인 선택이었습니다.;

피해야 할 답변

- 유명해서 골랐다.;
- 다들 많이 쓰길래 골랐다.;

10-1. 왜 Firebase 같은 NoSQL이 아니라 PostgreSQL이었나요?

좋은 답변

이 프로젝트는 실시간 채팅이나 문서 저장보다 거래, 카테고리, 계좌, 예산 간 관계와 집계가 더 중요했습니다. 참조 무결성, 월별 집계, 카테고리별 합계 같은 요구가 핵심이어서 스키마리스 저장소보다 관계형 DB가 더 자연스럽다고 판단했습니다.;

피해야 할 답변

- NoSQL은 어려워 보여서 안 썼다.;
- 그냥 SQL이 더 익숙할 것 같았다.;

10-2. 왜 Prisma가 아니라 Drizzle이었나요?

좋은 답변

`Prisma`의 장점은 분명 있지만, 당시에는 TypeScript 코드 안에서 스키마와 타입을 함께 관리하는 단순성이 더 중요했습니다. `Drizzle`은 스키마 정의 자체가 타입 역할을 하기 때문에 generate 단계 없이 개발 흐름을 가져가기 좋았고, 제 학습 단계에서도 더 직관적이었습니다.;

피해야 할 답변

- Prisma는 무겁다고만 들었다.;
- 그냥 더 마이너한 걸 써보고 싶었다.;

10-3. 왜 NextAuth가 아니라 Better Auth였나요?

좋은 답변

`NextAuth` 는 표준성과 생태계가 강한 장점이 있지만, 이 프로젝트에서는 기존 DB 구조와 인증 로직을 더 자연스럽게 붙이는 것이 중요했습니다. `Better Auth` 는 가입 직후 기본 카테고리 시딩이나 세션 저장 전 최소화 처리 같은 훅을 인증 흐름에 직접 붙이기 쉬워서 현재 프로젝트엔 더 잘 맞았습니다.;

피해야 할 답변

- Better Auth가 더 최신이라서 골랐다.;
- NextAuth는 복잡해 보여서 피했다.;

---

11. 초보자라서 안전한 기술만 고른 것 아닌가요?

좋은 답변

일부는 맞습니다. 다만 그걸 단순히 소극적 선택이라고 보진 않습니다. 처음 다루는 영역일수록, 도메인 적합성과 학습 가능성, 실패 확률을 같이 보는 것이 더 책임 있는 판단이라고 생각합니다. 회사에서도 무조건 새롭거나 화려한 기술보다 팀과 도메인에 맞는 검증된 선택이 더 중요하다고 봅니다.;

---

12. AI 파싱이 핵심인데, 신뢰성이 떨어지면 서비스 자체가 무너지는 것 아닌가요?

좋은 답변

그래서 AI를 자동 확정 엔진이 아니라 입력 마찰을 줄이는 보조 계층으로 봤습니다. `LLM` 출력은 확률적이기 때문에 오파싱 가능성이 있고, 금액, 날짜, 카테고리 같은 핵심 필드는 사용자가 확인하고 수정할 수 있어야 합니다. 목표는 100% 자동화보다 입력 부담을 유의미하게 줄이는 것이었습니다.;

12-1. 왜 LLM provider를 여러 개 붙였나요?

좋은 답변

프로덕션 관점에서는 하나로 단순화할 수도 있지만, 이 프로젝트는 학습과 실험 성격도 있었습니다. 짧은 텍스트와 긴 텍스트, 이미지 입력의 특성이 달랐고, 장애나 속도 이슈에 대비하려면 provider를 교체 가능하게 설계하는 것이 유리했습니다. 중요한 건 여러 개를 썼다는 사실보다, 인터페이스를 통일하고 폴백 기준을 명확히 둔 점입니다.;

피해야 할 답변

- 그냥 이것저것 붙여보고 싶었다.;
- 많을수록 좋아 보인다고 생각했다.;

12-2. 텍스트 입력에 base64 위장 payload를 왜 따로 막았나요?

좋은 답변

공개 텍스트 입력 필드에 이미지 `base64` 같은 대형 payload를 직접 넣으면 `LLM` 토큰을 과도하게 소모시키는 공격이 가능하기 때문입니다. 그래서 길이만 보는 게 아니라, `base64` 형태 자체를 탐지해 비정상 입력을 먼저 차단하는 것이 필요했습니다.;

피해야 할 답변

- 혹시 몰라서 그냥 막았다.;
- 길이만 길면 다 막으면 된다고 생각했다.;

12-3. 이미지 입력은 왜 MIME, 바이트, base64를 다 따로 검증했나요?

좋은 답변

클라이언트가 보내는 길이 정보는 위조될 수 있기 때문입니다. MIME 타입 화이트리스트, 원본 바이트 크기 제한, `base64` 길이 교차 검증, 실제 디코딩 가능 여부까지 봐야 서버가 독립적으로 입력을 신뢰하지 않는 구조가 됩니다.;

피해야 할 답변

- 프론트에서도 검사하니까 서버는 대충 봐도 된다.;
- 파일 크기만 보면 충분하다.;

12-4. IP와 UserAgent를 왜 해싱하거나 최소화했나요?

좋은 답변

인증과 보안 이벤트를 추적하려면 식별자는 필요하지만, 원문 IP나 UserAgent를 그대로 저장하면 개인정보 노출 리스크가 커집니다. 그래서 동일성 비교는 가능하되 원문을 직접 복원하기 어렵게 최소화한 것입니다.;

피해야 할 답변

- 보안이라서 그냥 해싱했다.;
- 법적으로 문제될까 봐 막연히 줄였다.;

12-5. LLM이 JSON이 아닌 이상한 응답을 주면 어떻게 하나요?

좋은 답변

실제 `LLM` 은 코드블록, 설명문, 배열, `<think>` 같은 형식을 섞어 줄 수 있기 때문에, 단순 `JSON.parse()` 로 끝낼 수 없습니다. 응답 정제, JSON 영역 추출, 거부 응답 처리, 타입 검증 순서로 방어적으로 파싱해야 안정성이 올라갑니다.;

피해야 할 답변

- 프롬프트에 JSON만 달라고 적었으니 괜찮다.;
- 실패하면 다시 호출하면 된다.;

---

13. AI 코드 비중이 높다는데, 그럼 본인 실력은 무엇으로 증명하나요?

좋은 답변

이 프로젝트는 구현 단계에서 AI의 도움을 많이 받았지만, 저는 문제 정의, 구조 선택, 기술 채택, 미도입 이유, 검증 기준, 최종 승인 책임을 맡았습니다. 손코딩 비율보다 중요한 건 왜 이 구조를 택했고 어떤 리스크를 감수했는지 설명할 수 있느냐라고 생각합니다.;

피해야 할 답변

- 거의 AI가 다 했다.;
- 코드 한 줄 한 줄은 잘 모른다.;

---

14. 에이전트 스킬과 파이프라인 문서는 왜 만든 건가요?

좋은 답변

AI에게 일을 떠넘기기 위한 장치라기보다, AI를 써도 승인, 리뷰, QA, 보안 체크를 빠뜨리지 않도록 제 작업 기준을 문서화한 것입니다. 혼자 작업하면 설계와 구현과 검증이 한 흐름 안에서 섞이기 쉬워서, 역할을 분리해 체크포인트를 강제하려고 했습니다.;

---

15. 혼자 하는 프로젝트에 PM, reviewer, QA까지 두는 건 과한 것 아닌가요?

좋은 답변

과해 보일 수 있다는 점은 이해합니다. 다만 의도는 역할놀이가 아니라 검증 분리였습니다. 특히 AI가 작성한 코드 비중이 높을수록 구현과 검토를 한 흐름으로 처리하면 놓치는 게 많아져서, 설계와 검증 기준을 나눠 관리하려고 했습니다.;

---

16. Capacitor까지 붙인 건 너무 욕심 아닌가요?

좋은 답변

위험이 있었다는 점은 맞습니다. 다만 가계부는 모바일 사용 빈도가 높아서, 하나의 웹 코드베이스로 앱 확장성까지 검증해보는 것이 현실적이라고 판단했습니다. 완전 네이티브보다 개발 속도와 코드 재사용성 측면에서 더 적절했습니다.;

---

17. Capacitor 때문에 웹 품질이 희생된 것 아닌가요?

좋은 답변

일부는 맞습니다. 앱형 UX를 우선하면서 웹접근성, 브라우저 친화성, SEO와 긴장이 생긴 건 사실입니다. 다만 그 경험 덕분에 앱형 UX와 웹 표준 품질 사이의 트레이드오프를 더 선명하게 이해하게 됐고, 에이전시 프로젝트에서는 웹 기준을 훨씬 더 앞세워야 한다는 점도 명확해졌습니다.;

---

18. Vue/Svelte 중심이었는데 왜 React/Next 학습을 실제 제품과 같이 묶었나요?

좋은 답변

실제 문제를 풀어야 학습이 더 깊어진다고 판단했습니다. 다만 그만큼 리스크가 있었기 때문에, 오히려 `Zustand`, `React Query` 같은 계층을 일부러 덜어내고 학습 범위와 구현 범위를 동시에 통제하려고 했습니다.;

---

19. 프론트엔드 관점에서 API 구조와 데이터 흐름을 이해한다고 했는데, 이 프로젝트에서 그게 어디서 드러나나요?

좋은 답변

거래 수정 하나가 거래 목록뿐 아니라 통계, 예산, 자산에 동시에 영향을 준다는 점을 구조로 다뤘다는 데서 드러납니다. 저는 화면만 그린 게 아니라 저장, 재검증, 인증, user scope, 파생 데이터 정합성을 하나의 흐름으로 보려고 했습니다.;

---

20. 다시 한다면 제일 먼저 무엇을 바꾸겠습니까?

좋은 답변

에이전시 면접 기준으로는 접근성과 시맨틱 구조를 가장 먼저 바꾸겠습니다. 이유는 이건 뒤에 붙일수록 비용이 커지고, 컴포넌트 구조와 UX 기준이 같이 흔들리기 때문입니다. 그 다음이 메타데이터와 공개 페이지 구조, 그리고 필요 시 클라이언트 캐시 전략 보완입니다.;

마지막 정리

압박 질문에서 중요한 건 `완벽했다` 가 아니다.;

> 약점을 숨기지 않고, 왜 그런 선택을 했는지 설명하고, 그 선택으로 무엇을 얻고 잃었는지 말한 뒤, 다음 판단 기준까지 명확히 제시하는 것;